

TECHNICAL AUDIT

VIXOR Platform

Comprehensive Security, Performance & Infrastructure Review

Document Classification	Confidential
Audit Scope	Full-Stack: API, Database, Auth, Infrastructure
Platform	Vercel Serverless (h3/Nitro) + Supabase + Upstash Redis
Endpoints Reviewed	15
Database Tables	32
Date	July 2026

Generated by automated audit pipeline. All findings are based on static analysis, dependency review, and architectural assessment.

Table of Contents

Table of Contents	2
1. Executive Summary	4
Overall Scores	4
Key Findings Summary	4
2. API Security Audit	6
Endpoint Security Matrix	6
API Security Problems	7
3. Database & Supabase Audit	8
Database Schema Overview	8
Database Problems	8
4. Authentication & Authorization Audit	10
Auth Flow Comparison	10
Auth Problems	10
5. Performance Audit	12
Performance Metrics	12
Performance Problems	12
6. Monitoring & Logging Audit	14
Monitoring Infrastructure	14
Monitoring Problems	15
7. Rate Limiting & Resilience Audit	16
Rate Limiting Configuration	16
Resilience Problems	16
8. Build & Bundle Analysis	18
Build Configuration	18
Build Problems	19
9. Deployment & CI/CD Audit	20
CI/CD Status	20
CI/CD Problems	21
10. Immediate Fixes & Future Improvements	22

Immediate Fixes (1-2 Weeks)	22
Short-Term Improvements (1-3 Months)	22
Long-Term Strategic Initiatives	22
Priority Action Matrix	23
Audit Conclusion	23

1. Executive Summary

This technical audit provides a comprehensive evaluation of the VIXOR cryptocurrency trading intelligence platform. The assessment covers 15 API endpoints, 32 Supabase database tables, authentication flows, caching infrastructure, rate limiting mechanisms, build pipelines, and deployment practices. VIXOR operates as a full-stack application deployed on Vercel serverless functions using the h3/Nitro runtime on Node.js 22.x, backed by Supabase for persistent storage and Upstash Redis for caching and rate limiting.

The platform serves a complex user base spanning Telegram WebApp users, wallet-based signers, and standard browser-based traders. It integrates multiple AI providers (OpenAI, Anthropic, Groq, ZAI) for copilot streaming, maintains real-time WebSocket connections to Binance and DexScreener for live market data, and runs scheduled background jobs via Vercel cron for signal generation and alert monitoring. The application bundle is built with Vite 7.3.1 and TanStack Start 1.168.25, employing post-build bundle patching for Vercel compatibility.

Overall, the platform demonstrates solid architectural foundations with well-structured domain modules, a resilient caching layer, and thoughtful security headers. However, the audit identified critical gaps in test coverage, monitoring and alerting, CI/CD automation, and error boundary coverage. These systemic weaknesses pose operational risks that could compound under load or during incidents. The absence of automated testing is the single most concerning finding, as it leaves every deployment unverified against regressions.

Overall Scores



Overall Weighted Score: 5.4 / 10 The platform scores well on security fundamentals and rate limiting but falls significantly short on operational readiness. The lack of automated tests, CI/CD pipelines, and a monitoring/alerting system represent existential risks for production reliability.

Key Findings Summary

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
SEC-01	P0	X-Admin-Key exposed in Vercel cron routes without IP allowlisting	Unauthorized cron invocation	Critical	vercel.json, server/api/*	Missing defense-in-depth
MON-01	P0	No monitoring, alerting, or incident response system deployed	Silent failures in production	Critical	Infrastructure	No operational tooling

CI-01	P0	Zero automated tests, no CI/CD pipeline for any deployment	Unverified deploys, regressions	Critical	Entire codebase	No test infrastructure
PERF-01	P1	In-memory LRU cache inconsistent across serverless instances	Cache misses, stale data	High	shared/cache.ts, resilience/*	Stateful design on stateless infra
AUTH-01	P1	Wallet nonce challenge-response lacks expiry enforcement	Replay attacks possible	High	wallet/connect.ts, wallet/session.ts	Missing TTL on nonce

2. API Security Audit

The VIXOR API surface comprises 15 endpoints deployed as Vercel serverless functions using the h3/Nitro runtime. Each endpoint runs on Node.js 22.x in an isolated serverless container with a 10-second execution timeout for standard functions and 60 seconds for cron-based routes. The API handles sensitive operations including authentication, financial data retrieval, AI-powered analysis streaming, wallet connection management, and Telegram webhook processing.

Security headers are properly configured via the `vercel.json` configuration, including Content-Security-Policy (CSP), Cross-Origin Resource Sharing (CORS) restricted to `vixor.app` and `vercel.app` origins, and standard security headers such as X-Frame-Options, X-Content-Type-Options, and Referrer-Policy. The CORS whitelist prevents unauthorized cross-origin requests from third-party domains, which is a positive security measure. However, the allowlist includes all `vercel.app` subdomains, which could enable preview deployment abuse by unauthorized actors who can create Vercel projects.

The most critical finding in the API security audit concerns the X-Admin-Key mechanism used to authenticate Vercel cron invocations. While the key provides basic authentication, it lacks defense-in-depth protections such as IP allowlisting, request signing, or key rotation. Anyone who discovers the static header value could invoke cron endpoints directly, triggering signal generation or alert checks outside their intended schedule. This is exacerbated by the fact that the key is stored as an environment variable with no rotation policy.

Endpoint Security Matrix

Endpoint	Auth	RL	Input Val	Status
copilot-stream	JWT	20/min	Yes	OK
generate-signals	Admin Key	None	N/A	At Risk
check-alerts	Admin Key	None	N/A	At Risk
wallet/connect	Public	30/min	Partial	Warning
wallet/session	JWT	20/min	Yes	OK
telegram-webhook	Public	30/min	Partial	Warning
market-overview	Optional	120/min	Yes	OK
discover	JWT	120/min	Yes	OK
sol-price	Public	120/min	Yes	OK
health	None	None	N/A	OK
metrics	Admin Key	None	N/A	At Risk
stars-webhook	Public	30/min	Partial	Warning
p1-validate	None	None	Yes	Info

migrate	Admin Key	None	N/A	At Risk
reanalysis-cron	Admin Key	None	N/A	At Risk

RL = Rate Limit. Auth = Authentication method. Input Val = Input validation. 5 endpoints use Admin Key without IP allowlisting.

API Security Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
SEC-01	P0	Admin-Key cron endpoints lack IP allowlisting and request signing	Cron endpoints can be invoked by any attacker who discovers the header value	Critical	vercel.json, server/api/generate-signals.ts, server/api/check-alerts.ts	Static secret without defense-in-depth
SEC-02	P1	CORS allows all vercel.app subdomains including adversarial preview deployments	Preview deployments could bypass CORS for API abuse	High	vercel.json	Overly permissive origin wildcard
SEC-03	P1	Webhook endpoints (telegram, stars) lack payload signature verification	Forged webhook payloads could inject fake data or commands	High	server/api/telegram-webhook.ts, server/api/stars-webhook.ts	Missing cryptographic verification
SEC-04	P1	No request size limits on streaming SSE endpoint (copilot-stream)	Oversized payloads could exhaust serverless memory and timeout	High	server/api/copilot-stream.ts	No body size enforcement
SEC-05	P2	Health endpoint exposes internal metrics without authentication	System information leakage aids reconnaissance	Medium	server/api/health.ts	Overly verbose public endpoint

API Security Score: 7 / 10 - Solid fundamentals with CSP and CORS, but critical gaps in admin endpoint hardening and webhook verification.

3. Database & Supabase Audit

The VIXOR database layer is built entirely on Supabase, which provides a managed PostgreSQL instance with built-in Row-Level Security (RLS), real-time subscriptions, and auto-generated TypeScript types. The schema currently spans 32 tables covering user management, trading signals, price alerts, trade journals, copilot conversations, user memories, wallet sessions, broker connections, payment records, arbitrage data, and experiment tracking. The auto-generated types file alone contains 1654 lines of TypeScript definitions, reflecting the complexity of the data model.

The dual-client pattern is a notable architectural choice: the browser client uses the Supabase JS SDK with Row-Level Security policies for user-scoped data access, while the server-side admin client bypasses RLS using the service role key for unrestricted database operations. This separation is correctly implemented, and the service role key is properly restricted to server-side code paths. However, the admin client is used in multiple server functions without centralized access control, increasing the blast radius if any single endpoint is compromised.

Row-Level Security is enabled on sensitive tables including `daily_signals`, `trades`, `user_memories`, and `price_alerts`. However, the audit found that several newer tables added during rapid feature development may lack RLS policies entirely. The migration history shows a pattern of enabling RLS as a separate migration step after table creation, which creates a window where data is exposed without row-level filtering. Additionally, the auto-generated types file at 1654 lines suggests heavy coupling between the application and the database schema, making schema migrations riskier without a comprehensive test suite.

Database Schema Overview

Total Tables	32
TypeScript Types	1,654 lines (auto-generated)
RLS Enabled Tables	18 (confirmed), 14 uncertain
Migration Count	26 SQL migration files
Client Pattern	Dual (Browser RLS + Admin Service Role)
Database Provider	Supabase (Managed PostgreSQL)
Realtime Subscriptions	Enabled for price data

Database Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
----	----------	---------	--------	------	----------------	------------

DB-01	P0	14 tables have unconfirmed RLS status; newer tables may lack row-level security entirely	Unauthorized data access if RLS is missing on any table	Critical	supabase/migrations/*	RLS added as afterthought in separate migration
DB-02	P1	1654-line auto-generated types create tight schema coupling; no migration safety net	Schema changes can break application silently	High	src/shared/supabase/types.ts	Direct dependency on auto-generated output
DB-03	P1	No database-level audit logging or change tracking enabled	Undetected data tampering or unauthorized modifications	High	Supabase project settings	Missing audit infrastructure
DB-04	P1	Admin service role key used broadly across server functions without centralized guard	Compromised endpoint grants full database access	High	server/api/*, src/shared/supabase/client.server.ts	Scattered admin client usage
DB-05	P2	No foreign key constraints documented; referential integrity assumed	Orphaned records, data corruption on cascading deletes	Medium	supabase/migrations/*	Missing FK documentation
DB-06	P2	No database connection pooling configuration verified	Connection exhaustion under high concurrency	Medium	Supabase project config	Default settings may be insufficient

Database Score: 7 / 10 - Good use of RLS and dual-client pattern, but unverified RLS coverage on newer tables is a critical gap.

4. Authentication & Authorization Audit

VIXOR implements three distinct authentication pathways to serve its diverse user base. The primary method leverages Supabase's built-in JWT authentication via the `getUser()` API, which validates session tokens issued by Supabase Auth. This provides a robust, well-tested authentication foundation with automatic token refresh, session management, and support for multiple social providers. The JWT tokens carry user claims that are verified server-side on every protected endpoint, ensuring that only authenticated users can access sensitive operations such as trade management, copilot conversations, and personalized signal delivery.

The second authentication method supports Telegram WebApp users through an auto-signin flow. When a user opens VIXOR from within Telegram, the WebApp SDK provides an `init_data` payload that is validated server-side to extract the user's Telegram identity. This flow creates or links a Supabase user account automatically, enabling seamless access without requiring separate credentials. While convenient, this auto-signin mechanism requires careful validation of the Telegram `init_data` signature to prevent impersonation attacks, particularly if the validation logic does not check payload freshness or Telegram bot token rotation.

The third authentication pathway implements a wallet-based challenge-response mechanism for Web3 users. When a wallet attempts to connect, the server generates a cryptographic nonce stored in Redis, presents it to the client, and requires the wallet to sign it with its private key. The signed response is verified against the original nonce to authenticate the wallet address. This is a standard Web3 authentication pattern, but the audit found that the nonce lacks an explicit expiry TTL enforcement, meaning stale nonces could theoretically be reused in replay attacks if an attacker intercepts the signed response. While Redis key expiration provides implicit TTL, the application layer should enforce a shorter, explicit nonce validity window.

Auth Flow Comparison

JWT (Supabase)	Mature, token refresh, multi-provider support, session management
Telegram WebApp	Auto-signin via <code>init_data</code> , seamless UX, requires signature validation
Wallet Challenge	Nonce in Redis, challenge-response, standard Web3 pattern, needs TTL
Session Storage	Supabase sessions + Redis nonces for wallet auth
Token Verification	Server-side <code>getUser()</code> on protected endpoints
Authorization Model	Row-Level Security (database) + API middleware

Auth Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
----	----------	---------	--------	------	----------------	------------

AUTH-01	P1	Wallet nonce lacks explicit expiry enforcement beyond Redis TTL	Stale nonces could enable replay attacks with intercepted signatures	High	server/api/wallet/connect.ts, server/api/wallet/session.ts	No application-level nonce TTL check
AUTH-02	P1	Telegram init_data validation may not check payload timestamp freshness	Replayed Telegram auth payloads could impersonate users	High	src/domains/user/server/telegram-verify.ts	Missing timestamp/freshness validation
AUTH-03	P1	No session revocation mechanism for compromised JWT tokens	Compromised sessions persist until natural expiry	High	src/shared/supabase/auth-middleware.ts	No token blacklist or revocation endpoint
AUTH-04	P2	Wallet-to-user account linking has no consent verification step	Unauthorized wallet linking to existing accounts	Medium	server/api/wallet/connect.ts	Missing user consent flow
AUTH-05	P2	No brute-force protection on Telegram auto-signin endpoint	Enumeration of Telegram user IDs via repeated attempts	Medium	server/api/telegram-webhook.ts	Missing attempt throttling

Authentication Score: 6 / 10 - Solid multi-pathway auth, but missing session revocation, nonce expiry, and brute-force protections.

5. Performance Audit

Performance is a critical concern for VIXOR given its real-time market data requirements, AI streaming workloads, and serverless deployment model. The platform must simultaneously handle WebSocket connections for live prices, server-sent events (SSE) for AI copilot responses, and traditional HTTP requests for data retrieval. The build toolchain produces a client-side bundle via Vite 7.3.1 and TanStack Start 1.168.25, with post-build patching applied to ensure Vercel serverless compatibility. The caching strategy employs a hybrid approach combining Upstash Redis for distributed caching with an in-memory LRU cache for local performance.

The hybrid caching design is architecturally sound but contains a fundamental inconsistency when deployed on serverless infrastructure. Vercel serverless functions are ephemeral and stateless; each invocation may run in a completely different container instance. The in-memory LRU cache provides no benefit in this environment because its contents are lost when the container is recycled. In practice, this means the in-memory cache layer introduces unnecessary complexity without delivering performance gains, while creating a false sense of caching consistency. Furthermore, the Redis cache layer requires network round-trips on every cold start, which can add 50-200ms of latency to cache-backed operations.

The N+1 query pattern is a potential concern given the data-heavy nature of the application. Several endpoints fetch signal lists, trade histories, or alert collections that then require per-item queries for enrichment data such as token prices, user preferences, or notification settings. Without a systematic query analysis or ORM-level eager loading, these per-item queries can multiply the database load significantly. The auto-generated Supabase types at 1654 lines suggest a complex schema where joins and relations are common, making N+1 prevention especially important. Additionally, the real-time WebSocket connections to Binance and DexScreener consume persistent resources that are not metered or limited, potentially causing resource exhaustion on long-lived serverless connections.

Performance Metrics

Build Tool	Vite 7.3.1 + TanStack Start 1.168.25
Bundle Post-Processing	Custom patching script for Vercel compatibility
Caching Layers	Upstash Redis (distributed) + In-memory LRU (local)
Real-time Connections	Binance WS + DexScreener WS
AI Streaming	SSE (Server-Sent Events) for copilot-stream
Serverless Cold Start	Estimated 50-200ms for Redis-backed operations
Cache Inconsistency Risk	High (in-memory LRU on ephemeral containers)

Performance Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
PERF-01	P1	In-memory LRU cache is ineffective on Vercel serverless ephemeral containers	Wasted memory, false caching consistency, no performance benefit	High	src/shared/cache.ts, src/shared/resilience/lru-cache.ts	Stateful design on stateless infrastructure
PERF-02	P1	Potential N+1 queries in signal/trade list endpoints with per-item enrichment	Linear database load growth with data volume	High	src/domains/trades/functions.ts, src/domains/analysis/functions.ts	Missing eager loading or batch queries
PERF-03	P1	Post-build bundle patching adds deployment fragility and delays	Builds may break silently after Vite/TanStack updates	High	scripts/fix-vercel-bundle.mjs, vite.config.ts	Fragile string replacement approach
PERF-04	P2	WebSocket connections have no reconnection backoff or max retry limits	Connection storms after network disruptions	Medium	src/shared/market-data/binance-ws.ts, dexscreener-ws.ts	Missing backoff strategy
PERF-05	P2	No bundle size budget enforcement or size regression detection	Uncontrolled bundle growth over time	Medium	vite.config.ts	No size-limit plugin configured
PERF-06	P2	Redis cache has no stale-while-revalidate strategy for stale data tolerance	Users see errors during cache cold-start periods	Medium	src/shared/resilience/lru-cache.ts	Missing SWR pattern

Performance Score: 5 / 10 - Fundamental mismatch between stateful caching and serverless architecture, plus N+1 and bundle risks.

6. Monitoring & Logging Audit

The monitoring and logging landscape of VIXOR represents the most significant gap in the platform's operational readiness. While the platform implements structured JSON logging (JSONL format compatible with Loki/ELK stacks) and has Sentry initialized for error tracking, the absence of a comprehensive monitoring and alerting system means that production issues are likely discovered reactively rather than proactively. There is no dashboard for real-time system health, no automated alerting for error rate spikes, latency degradation, or resource exhaustion, and no incident response procedures documented or automated.

Sentry is initialized within the application for error capture, which provides stack traces, breadcrumbs, and release tracking for client-side and server-side errors. However, the audit found that Sentry's effectiveness is significantly undermined by incomplete error boundary coverage. React error boundaries are only partially implemented through the `RouteErrorBoundary` component, meaning that many component-level errors will propagate uncaught to the global error handler, losing valuable rendering context. Additionally, server-side errors in API routes are captured but not categorized by severity or impact, making it difficult to prioritize responses during incidents.

The structured JSON logger outputs JSONL-formatted logs suitable for ingestion by Loki or ELK-based log aggregation systems. This is a good foundational choice for observability, as it enables structured querying, log correlation across services, and integration with alerting rules. However, without a deployed log aggregation backend, these structured logs are effectively writing to stdout on ephemeral serverless containers that are recycled after each invocation. The logs are lost unless Vercel's built-in log retention is configured, which typically only retains logs for a limited time window. Mixpanel is initialized for product analytics, providing event tracking for user behavior, but this does not substitute for infrastructure monitoring.

Monitoring Infrastructure

Error Tracking	Sentry (initialized, partial error boundary coverage)
Structured Logging	JSONL format (JSON logger for Loki/ELK compatibility)
Analytics	Mixpanel (product event tracking)
Log Aggregation	None deployed (stdout on ephemeral containers)
Alerting System	None deployed (no PagerDuty, OpsGenie, or equivalent)
Uptime Monitoring	None deployed (no synthetic probes or health checks)
Dashboards	None deployed (no Grafana, Datadog, or equivalent)
Incident Response	No documented procedures or runbooks

Monitoring Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
MON-01	P0	No monitoring, alerting, or incident response system deployed in production	Silent failures, extended outage detection times, revenue loss	Critical	Infrastructure	No operational tooling investment
MON-02	P0	Structured JSONL logs lost on ephemeral serverless containers without aggregation	No forensic capability for debugging production issues	Critical	src/shared/structured-logger.ts, src/shared/logger.ts	No log aggregation backend
MON-03	P1	Sentry error boundary coverage is incomplete; only RouteErrorBoundary implemented	Component-level errors lost without rendering context	High	src/components/vixor/RouteErrorBoundary.tsx	Missing granular error boundaries
MON-04	P1	No API latency tracking, p99/p95 monitoring, or SLA measurement	Performance degradation invisible to operators	High	server/api/*	Missing latency instrumentation
MON-05	P2	Mixpanel analytics tracked but no product analytics dashboard configured	User behavior insights require manual Mixpanel exploration	Medium	src/shared/analytics.ts	No dashboard configuration
MON-06	P2	No synthetic uptime monitoring or external health check probing	Outages detected only when users report them	Medium	Infrastructure	Missing external monitoring

Monitoring Score: 3 / 10 - The weakest area of the audit. Logging foundation exists but nothing is deployed for visibility.

7. Rate Limiting & Resilience Audit

VIXOR implements a Redis-backed sliding window rate limiting system using Upstash Redis as the distributed counter store. This is a well-chosen approach for serverless environments where in-memory rate limiting is ineffective across instances. The rate limiting middleware applies tiered limits: a global rate of 120 requests per minute for general endpoints, 20 requests per minute for AI-intensive endpoints like copilot-stream, and 30 requests per minute for webhook endpoints. These limits are enforced through middleware that checks the sliding window counter before processing each request.

The sliding window algorithm provides smoother rate limiting compared to fixed-window approaches, as it prevents the burst-at-boundary problem where a client can send 2x the allowed requests by timing requests at window edges. The per-user rate limiting for AI endpoints is particularly important, as LLM API calls are expensive and have their own provider-level rate limits. By throttling at the application layer, VIXOR prevents individual users from consuming disproportionate AI resources. The implementation correctly returns 429 Too Many Requests responses with appropriate headers, enabling clients to implement backoff behavior.

However, the rate limiting implementation has several notable gaps. Admin-key authenticated endpoints (generate-signals, check-alerts, metrics, migrate, reanalysis-cron) have no rate limiting applied, which is acceptable for cron invocations but creates a vulnerability if the admin key is compromised. The resilience layer includes a circuit breaker implementation, but its configuration and coverage are unclear from the codebase. There is no evidence of retry logic with exponential backoff for failed external API calls (Binance, DexScreener, LLM providers), which means transient failures result in direct errors rather than graceful degradation. The background cron jobs lack dead-letter queues or failure notifications, so a failed signal generation run goes unnoticed.

Rate Limiting Configuration

Implementation	Redis-backed sliding window (Upstash)
Global Limit	120 requests/minute
Per-User AI Limit	20 requests/minute
Webhook Limit	30 requests/minute
Response Code	429 Too Many Requests with headers
Circuit Breaker	Implemented (coverage unclear)
Retry Logic	Not implemented for external APIs
Dead Letter Queue	Not implemented for cron jobs

Resilience Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
RES-01	P1	Admin-key endpoints have no rate limiting, amplifying key compromise impact	Unlimited requests if admin key is leaked	High	server/utls/with-rate-limit.ts	Rate limiting skipped for admin routes
RES-02	P1	No retry with exponential backoff for external API calls (Binance, DexScreener, LLM)	Transient failures cause direct user-facing errors	High	src/shared/market-data/*, src/shared/llm/*	Missing retry middleware
RES-03	P1	Cron job failures have no dead-letter queue or notification mechanism	Failed signal generation or alert checks go unnoticed	High	server/api/generate-signals.ts, server/api/check-alerts.ts	No failure notification pipeline
RES-04	P2	Circuit breaker state not shared across serverless instances via Redis	Each instance has independent breaker state	Medium	src/shared/resilience/circuit-breaker.ts	In-memory state on ephemeral containers
RES-05	P2	No graceful degradation or fallback responses for upstream failures	Users see raw errors when external services fail	Medium	server/api/market-overview.ts, discover.ts	Missing fallback patterns

Rate Limiting Score: 7 / 10 - Well-implemented sliding window with tiered limits, but resilience patterns are incomplete.

8. Build & Bundle Analysis

The VIXOR frontend is built using Vite 7.3.1 as the module bundler and TanStack Start 1.168.25 as the full-stack React framework. TanStack Start provides server-side rendering (SSR), file-based routing, and type-safe API routes, making it a modern and capable foundation. However, the combination of these tools with Vercel's serverless deployment model requires post-build bundle patching to ensure compatibility. A custom script (`fix-vercel-bundle.mjs`) applies string replacements and modifications to the build output after Vite completes its bundling process.

The post-build patching approach is inherently fragile and represents a significant maintenance risk. Every upgrade to Vite, TanStack Start, or their dependencies could alter the internal structure of the build output, causing the patching script to fail silently or introduce subtle runtime errors. The fact that this patching is necessary suggests a compatibility gap between TanStack Start's output expectations and Vercel's serverless function requirements that should be resolved at the framework level rather than patched post-build. There is no validation step to verify that the patched bundle is functionally equivalent to the pre-patch output.

The build configuration lacks several performance optimizations that are standard for production deployments. There is no bundle size budget enforcement, meaning the JavaScript payload can grow unbounded as features are added. Code splitting is likely handled by TanStack Start's built-in route-based splitting, but there is no evidence of manual chunk optimization for heavy dependencies like TradingView charting libraries, the AI streaming client, or the analytics SDK. The Vite configuration should leverage `rollupOptions.manualChunks` to isolate these heavy dependencies into separate chunks that can be loaded on demand. Additionally, there is no tree-shaking verification or unused export detection in the build pipeline.

Build Configuration

Bundler	Vite 7.3.1
Framework	TanStack Start 1.168.25
Post-Build Patching	fix-vercel-bundle.mjs (string replacements)
Bundle Size Budget	Not configured
Code Splitting	Route-based (TanStack Start default)
Manual Chunks	Not configured
Tree-Shaking Verification	Not configured
Build Validation	No post-build functional tests
SSR	Enabled (TanStack Start server-side rendering)

Build Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
BLD-01	P1	Post-build bundle patching is fragile and breaks on dependency updates	Silent build failures or runtime errors after upgrades	High	scripts/fix-vercel-bundle.mjs, vite.config.ts	Framework-serverless compatibility gap
BLD-02	P1	No bundle size budget enforcement or regression detection	Uncontrolled JavaScript payload growth	High	vite.config.ts	Missing rollup-plugin-size-limit
BLD-03	P2	Heavy dependencies (TradingView, AI SDK, Analytics) not isolated into manual chunks	Large initial bundle load, slow first contentful paint	Medium	vite.config.ts	Missing manualChunks configuration
BLD-04	P2	No build-time type checking or linting in the build pipeline	Type errors and lint violations reach production	Medium	package.json, vite.config.ts	Missing tsc --noEmit in build script
BLD-05	P2	No source map generation strategy documented for production debugging	Production errors difficult to trace to source	Medium	vite.config.ts	Missing sourcemap configuration

Build & Bundle Score: 6 / 10 - Modern toolchain but post-build patching is a critical fragility point.

9. Deployment & CI/CD Audit

The VIXOR deployment pipeline represents the most operationally deficient area of the platform. There is no CI/CD pipeline whatsoever. No continuous integration checks run on code changes, no automated tests execute before deployment, and no deployment gates prevent broken code from reaching production. The current deployment workflow appears to rely on direct git pushes to the main branch, which trigger Vercel's built-in deployment mechanism. While Vercel provides preview deployments for pull requests, the absence of CI checks means these previews are not validated against any quality criteria before being promoted to production.

The absence of automated testing is the single most critical operational gap identified in this audit. With zero test coverage, every code change is effectively a blind deployment. There are no unit tests for utility functions and data transformations, no integration tests for API endpoints, no end-to-end tests for user flows, and no visual regression tests for the UI layer. The codebase does include a vitest configuration file and some test utility files, but the actual test coverage is negligible. Given the complexity of the platform with its 32 database tables, 15 API endpoints, 3 authentication pathways, real-time WebSocket connections, and multi-provider AI integration, the lack of automated verification is a significant risk multiplier for every deployment.

Vercel's deployment infrastructure itself is well-suited for the application architecture. Serverless functions provide automatic scaling, edge caching, and integrated CDN for static assets. The Vercel cron configuration correctly schedules the two background jobs (generate-signals at midnight, check-alerts at 00:30). However, without CI/CD, the deployment process lacks rollback automation, canary releases, and deployment verification. A failed deployment can only be detected after users are affected, and rollback requires manual intervention through the Vercel dashboard. There is no blue-green or canary deployment strategy, and no deployment pipeline that could enable progressive rollouts with automated rollback on error rate increase.

CI/CD Status

CI Pipeline	None (no GitHub Actions, no CircleCI, no Jenkins)
Automated Tests	Zero coverage (vitest configured but unused)
Code Quality Gates	None (no lint, no type-check, no bundle check)
Deployment Method	Git push to main triggers Vercel deploy
Preview Deployments	Vercel built-in (no validation)
Rollback Strategy	Manual via Vercel dashboard
Canary/Blue-Green	Not implemented
Environment Separation	Production only (no staging)

Deployment Verification

None (no smoke tests post-deploy)

CI/CD Problems

ID	Priority	Problem	Impact	Risk	Affected Files	Root Cause
CI-01	P0	Zero automated tests across the entire codebase (32 tables, 15 endpoints, 3 auth flows)	Every deployment is unverified; regressions reach production silently	Critical	Entire codebase	No test infrastructure or culture
CI-02	P0	No CI pipeline for any automated checks before deployment	No type checking, linting, or security scanning on code changes	Critical	Infrastructure	No CI/CD tooling configured
CI-03	P1	No staging environment for pre-production validation	All testing happens against production data and users	High	Infrastructure	Single-environment deployment
CI-04	P1	No deployment verification or smoke tests after Vercel deploy	Broken deployments detected only by user reports	High	vercel.json, Infrastructure	Missing post-deploy validation
CI-05	P1	No canary or blue-green deployment strategy for safe rollouts	Failed deployments affect 100% of users immediately	High	vercel.json, Infrastructure	Missing progressive deployment
CI-06	P2	No environment variable validation or secret rotation mechanism	Missing or invalid environment variables cause silent failures	Medium	.env.example, Vercel env config	No env validation step

CI/CD Score: 2 / 10 - Existential risk. Zero tests and no CI means every deploy is a leap of faith.

10. Immediate Fixes & Future Improvements

Based on the comprehensive audit findings across all nine areas, this chapter provides a prioritized action plan organized into immediate fixes (P0 issues requiring action within 1-2 weeks), short-term improvements (P1 issues for the next 1-3 months), and long-term strategic initiatives (P2 and architectural improvements for the next quarter). The prioritization considers both the severity of the finding and the effort required for remediation, ensuring that the highest-impact issues are addressed first with reasonable effort investment.

Immediate Fixes (1-2 Weeks)

The four P0 issues identified in this audit require immediate attention. First, the monitoring and alerting gap (MON-01, MON-02) should be addressed by deploying a lightweight monitoring solution such as UptimeRobot for external health checks and setting up Sentry alerts to route critical errors to a communication channel. Second, the testing gap (CI-01) should be mitigated by establishing a minimal test scaffold with at least API endpoint smoke tests and critical path unit tests, even if full coverage will take months to achieve. Third, the admin endpoint security (SEC-01) should be hardened by adding IP allowlisting to the Vercel cron configuration, restricting admin-key endpoints to Vercel's known cron IP ranges. Fourth, the database RLS verification (DB-01) should be completed by auditing all 32 tables and confirming or enabling Row-Level Security policies on every table containing user-specific data.

Short-Term Improvements (1-3 Months)

Short-term improvements should focus on establishing CI/CD infrastructure, improving resilience patterns, and fixing authentication weaknesses. A GitHub Actions CI pipeline should be configured with type checking (tsc --noEmit), linting (eslint), bundle size checks, and automated test execution on every pull request. The wallet nonce authentication should be hardened with explicit TTL enforcement and the Telegram init_data validation should include timestamp freshness checks. A session revocation mechanism should be implemented for JWT tokens. External API calls should be wrapped with retry logic using exponential backoff with jitter. The in-memory LRU cache should be removed from the serverless deployment path or replaced with a pure Redis-backed cache. Webhook endpoints should implement payload signature verification for Telegram and Stars integrations.

Long-Term Strategic Initiatives

Long-term strategic initiatives should transform VIXOR from a functional prototype into a production-grade platform. A comprehensive test suite should be developed targeting 80%+ coverage across unit, integration, and end-to-end layers. The build pipeline should be refactored to eliminate the fragile post-build patching step, either by contributing upstream fixes to TanStack Start or by migrating to

a more Vercel-compatible framework configuration. A staging environment should be established with production-like data for pre-deployment validation. The monitoring stack should be expanded to include Grafana dashboards, distributed tracing with OpenTelemetry, and SLA-based alerting with PagerDuty or equivalent. The database layer should implement audit logging, connection pooling optimization, and a schema migration safety net with automated rollback capability.

Priority Action Matrix

Priority	Action	Effort	Impact	Timeline
P0	Deploy UptimeRobot + Sentry alert routing	1 day	Critical	Week 1
P0	Add IP allowlisting to cron endpoints	2 hours	Critical	Week 1
P0	Verify and enable RLS on all 32 tables	1 day	Critical	Week 1-2
P0	Create minimal test scaffold with API smoke tests	3 days	High	Week 2
P1	Set up GitHub Actions CI with tsc, lint, test	2 days	High	Month 1
P1	Add nonce TTL + Telegram timestamp validation	1 day	High	Month 1
P1	Remove in-memory LRU, use pure Redis cache	1 day	Medium	Month 1
P1	Add retry with backoff for external APIs	2 days	High	Month 2
P1	Implement webhook payload verification	2 days	High	Month 2
P2	Achieve 80%+ test coverage	4 weeks	High	Quarter
P2	Eliminate post-build bundle patching	2 weeks	Medium	Quarter
P2	Deploy Grafana + OpenTelemetry tracing	1 week	High	Quarter
P2	Establish staging environment	1 week	High	Quarter

Audit Conclusion

VIXOR demonstrates strong architectural foundations with well-structured domain modules, thoughtful security header configuration, an effective Redis-backed rate limiting system, and a sensible hybrid caching approach for distributed environments. The platform's multi-provider AI integration, real-time WebSocket connections, and dual Supabase client pattern reflect engineering maturity in the core design. However, the operational layer is critically underdeveloped: the absence of automated testing, CI/CD pipelines, monitoring/alerting, and staging environments creates a fragile deployment pipeline where every release carries significant risk.

The most urgent recommendation is to establish a minimal operational safety net within the first two weeks: deploy external uptime monitoring, add IP restrictions to admin endpoints, verify database security policies, and create a basic test scaffold. These four actions alone would dramatically reduce the platform's operational risk profile. Subsequently, building out CI/CD infrastructure and achieving

meaningful test coverage should be the top priority for the next quarter, as these foundational capabilities enable all other improvements with confidence. The VIXOR platform has the architectural strength to scale; what it needs now is the operational discipline to ship safely.